Vulnerabilidades de Segurança em Código Gerado por Inteligência Artificial: uma análise com base na OWASP Top 10

Felipe Barcelos
Curso de Ciência da Computação Centro Universitário de Brasília (UniCEUB)
Brasília-DF, 2026

Resumo

A geração de código por modelos de linguagem de grande escala tornou-se parte rotineira do desenvolvimento de software, o que desloca decisões de segurança para ferramentas que raramente recebem requisitos explícitos nesse sentido. Este trabalho revisa a evidência empírica acumulada desde 2021 sobre a incidência de vulnerabilidades em código produzido por essas ferramentas, tomando o OWASP Top 10 como referência de classificação. Os estudos analisados convergem para a estimativa de que cerca de 40% a 45% desse código contém falhas de segurança, proporção que se mantém estável mesmo à medida que os modelos ganham capacidade. Discutem-se as causas dessa persistência, os tipos de vulnerabilidade mais recorrentes, o risco emergente de alucinação de pacotes e as limitações das técnicas de mitigação atualmente disponíveis. Conclui-se que código gerado por inteligência artificial deve ser tratado como material de origem não confiável, sujeito a revisão de segurança antes de qualquer uso em produção.

1 Introdução

A adoção de assistentes de programação baseados em modelos de linguagem deixou de ser exceção e passou a constituir prática comum no desenvolvimento de software. Segundo o presidente-executivo do GitHub, Thomas Dohmke, quase metade do código escrito nos arquivos em que o Copilot está habilitado já é produzida pela própria ferramenta. Esse movimento, que Andrej Karpathy popularizou sob o rótulo de vibe coding, transfere para os modelos boa parte das decisões que antes cabiam ao programador, inclusive aquelas com implicações de segurança. O problema é que essas decisões costumam ser tomadas sem que qualquer requisito de segurança tenha sido comunicado ao modelo.

Diante desse cenário, cabe perguntar com que frequência, de que formas e por quais razões o código gerado por esses modelos introduz vulnerabilidades, e em que medida tais falhas podem ser detectadas e corrigidas de maneira sistemática. A pergunta tem relevância tanto científica quanto prática. Do lado científico, existe hoje um corpo de evidências consistente, construído desde 2021, que permite uma síntese fundamentada. Do lado prático, o código inseguro gerado por inteligência artificial já ingressa em produção em larga escala, muitas vezes de forma indireta, por meio de mantenedores de projetos abertos, fornecedores terceirizados e plataformas de baixo código. Mesmo organizações que não adotam essas ferramentas diretamente acabam expostas ao risco.

Este artigo organiza a evidência disponível em torno de duas referências complementares do OWASP e discute suas implicações para quem precisa avaliar a segurança desse tipo de código.

2 Dois referenciais do OWASP

A literatura sobre o tema exige a distinção entre dois documentos que, embora compartilhem a marca OWASP, tratam de objetos diferentes. O OWASP Top 10 de 2021 cataloga as vulnerabilidades clássicas de aplicações web, como quebra de controle de acesso, falhas criptográficas e as diversas formas de injeção. Ele descreve, portanto, as falhas que podem estar presentes no código que um modelo escreve. Já o OWASP Top 10 for LLM Applications, cuja edição estável de 2025 foi publicada pelo OWASP GenAI Security Project, trata dos riscos das aplicações que incorporam modelos de linguagem como componente de arquitetura, entre eles a injeção de prompt, o tratamento inadequado das saídas e as vulnerabilidades de cadeia de suprimentos.

A diferença é mais do que terminológica. No primeiro caso, o modelo é o produtor do código inseguro; no segundo, é uma peça da aplicação que pode ser atacada ou induzida a comportamentos indesejados. Os dois planos se tocam em pontos específicos, sobretudo na questão da alucinação de pacotes, que é simultaneamente um defeito do código gerado e um risco de cadeia de suprimentos. Para o estudo da segurança do código propriamente dito, interessam de perto as categorias do lado LLM ligadas à geração, em especial a de cadeia de suprimentos, a de tratamento das saídas e a de envenenamento de dados e do modelo.

3 A evidência empírica

3.1 A prevalência de código inseguro

O estudo que inaugurou essa linha de investigação foi conduzido por Pearce e colaboradores e apresentado no IEEE Symposium on Security and Privacy de 2022. Os autores submeteram o GitHub Copilot a 89 cenários e analisaram 1.689 programas resultantes com o CodeQL, complementado por inspeção manual. O resultado tornou-se referência obrigatória: aproximadamente 40% do código gerado continha vulnerabilidades. Mais revelador do que a média foi a variação por tipo de falha. Na categoria de injeção de comandos do sistema operacional, um dos cenários produziu programas vulneráveis em todas as vinte e uma sugestões oferecidas; na travessia de diretórios, todas as sugestões de maior pontuação eram vulneráveis. No extremo oposto, o cross-site scripting apresentou o melhor comportamento, com pouco menos de um quinto das sugestões afetadas.

Trabalhos posteriores não desmentiram esse quadro. A Veracode, em relatório de 2025 que testou mais de cem modelos em oitenta tarefas distribuídas por quatro linguagens, encontrou que 45% das amostras falharam nos testes de segurança. O dado mais incômodo desse relatório apareceu na atualização seguinte, quando a

empresa observou que a taxa de aprovação de segurança oscilava entre 45% e 55% independentemente da geração ou da data de lançamento do modelo, ao passo que a correção sintática do código subira de cerca de metade para praticamente a totalidade dos casos desde 2023. Em outras palavras, os modelos aprenderam a escrever código que funciona, mas não código que é seguro. O benchmark CyberSecEval, mantido pela Meta, chegou a conclusão convergente por caminho independente, registrando código vulnerável em torno de 30% dos casos de teste. A concordância entre metodologias tão diferentes — prompts sintéticos, benchmarks e análise de repositórios reais — é o que confere solidez à estimativa.

3.2 Por que a capacidade não melhora a segurança

Um achado recorrente e contraintuitivo é o de que modelos mais capazes não produzem, por isso, código mais seguro. O CyberSecEval observou que os modelos com melhor desempenho em programação tendiam a sugerir código inseguro com mais frequência, não menos. A explicação está na origem do problema. Modelos como o Codex, que serviu de base ao Copilot, foram treinados sobre enormes volumes de código público extraído do GitHub, material que carrega padrões inseguros de longa data. O exemplo canônico é o uso do algoritmo MD5 para o armazenamento de senhas: como exemplos assim continuam abundantes em repositórios abertos, o modelo aprende a reproduzi-los. A ferramenta gera, afinal, a continuação mais provável diante do que viu no treinamento, e não a mais segura segundo qualquer critério de engenharia. À medida que o modelo se torna mais competente em imitar o código que lhe serviu de exemplo, ele imita também os defeitos desse código com maior fidelidade.

3.3 O fator humano

A segurança do código gerado não depende apenas do modelo, mas também de como o programador se relaciona com ele. Perry e colaboradores conduziram, e apresentaram na ACM CCS de 2023, o primeiro estudo de usuário de larga escala sobre o tema. Os participantes que dispunham de um assistente de IA escreveram, de modo consistente, código menos seguro do que os que trabalharam sem essa ajuda, resultado especialmente nítido nas questões de criptografia e de injeção de SQL. O que agrava o quadro é que esses mesmos participantes tendiam a acreditar que seu código era mais seguro, efeito que os autores batizaram de falsa sensação de segurança. Um levantamento da Snyk reforça o ponto: mais de três quartos dos desenvolvedores consultados julgavam o código de IA mais seguro do que o humano, ainda que a maioria reconhecesse que sugestões inseguras eram comuns, e menos de um quarto usava ferramentas de análise de dependências para verificar essas sugestões antes de aproveitá-las.

3.4 A alucinação de pacotes

Entre os riscos mais recentes, o de maior potencial destrutivo é a chamada alucinação de pacotes. Spracklen e colaboradores, em trabalho apresentado no USENIX Security de 2025, testaram dezesseis modelos sobre 576 mil amostras em Python e JavaScript

e verificaram que 19,7% dos pacotes recomendados simplesmente não existiam, o que gerou mais de duzentos mil nomes distintos de dependências inexistentes. O perigo não está na inexistência em si, mas na regularidade com que os mesmos nomes falsos reaparecem: a maioria deles surgia em execuções repetidas do mesmo prompt, o que torna o comportamento previsível. Um atacante pode então identificar os nomes mais frequentemente alucinados, registrá-los nos repositórios oficiais com código malicioso e aguardar que desenvolvedores instalem essas dependências por confiarem na sugestão da ferramenta. A técnica ficou conhecida como slopsquatting. Réplicas de 2026 com modelos de fronteira mostraram queda expressiva nas taxas, para algo entre 5% e 6%, o que indica progresso, mas não a eliminação do problema.

4 Detecção e mitigação

A resposta metodológica mais defensável para medir e conter essas falhas não passa por construir um novo detector, e sim por orquestrar ferramentas de análise estática já consolidadas. As três mais úteis para esse fim se complementam por desenho. O CodeQL realiza análise semântica com rastreamento de fluxo de dados, capaz de seguir uma entrada até o ponto perigoso em que ela é utilizada, ao custo de exigir código completo e compilável. O Semgrep opera por correspondência de padrões, dispensa a modelagem completa da árvore sintática e, por isso, funciona bem sobre trechos incompletos, como os que os modelos costumam produzir. O Bandit especializa-se em Python e captura com agilidade os antipadrões clássicos da linguagem. Combinar os três sob uma regra de decisão por maioria, em que um achado só é aceito quando ao menos duas ferramentas concordam, reduz de maneira sensível os falsos positivos, estratégia que a literatura recomenda e emprega.

Convém reconhecer, porém, os limites da análise estática. Um estudo de verificação formal recente constatou que a esmagadora maioria das vulnerabilidades formalmente comprovadas passava despercebida pelo conjunto das ferramentas industriais, o que recomenda que a inspeção manual de uma amostra acompanhe sempre a triagem automatizada. Quanto à mitigação pela via dos prompts, os resultados são modestos. Instruções que pedem código seguro, sobretudo quando citam explicitamente as boas práticas do OWASP, chegam a reduzir de forma apreciável a incidência de falhas em alguns modelos, mas nenhuma formulação elimina o problema, e diversos trabalhos observam que a técnica tende a redistribuir as categorias de vulnerabilidade em vez de reduzir seu total. O reparo iterativo guiado por análise estática, por sua vez, corrige parte das falhas, embora possa introduzir outras no processo.

5 Considerações finais

A evidência reunida sustenta três conclusões. A primeira é que a presença de vulnerabilidades no código gerado por inteligência artificial é alta e estável, situando-se em torno de dois quintos do total, sem sinal de melhora com o avanço dos modelos. A segunda é que essa estabilidade decorre da própria natureza do treinamento, o que a

torna um problema estrutural e não uma limitação passageira. A terceira é que o fator humano amplifica o risco, na medida em que a confiança excessiva na ferramenta reduz a vigilância justamente quando ela seria mais necessária.

A implicação prática é direta. Código gerado por inteligência artificial deve ser tratado como material de origem não confiável. Toda sugestão precisa passar por revisão de segurança, por análise estática e pela verificação das dependências indicadas antes de alcançar qualquer ambiente de produção. Longe de dispensar o julgamento do programador, a difusão dessas ferramentas o torna mais indispensável do que antes.

Referências

ASARE, O.; NAGAPPAN, M.; ASOKAN, N. Is GitHub's Copilot as bad as humans at introducing vulnerabilities in code? Empirical Software Engineering, 2024. DOI: 10.1007/s10664-023-10380-1.

BHATT, M. et al. Purple Llama CyberSecEval: a secure coding benchmark for language models. arXiv:2312.04724, 2023.

FU, Y. et al. Security weaknesses of Copilot-generated code in GitHub projects: an empirical study. ACM Transactions on Software Engineering and Methodology, v. 34, n. 8, 2025. DOI: 10.1145/3716848.

KHOURY, R. et al. How secure is code generated by ChatGPT? arXiv:2304.09655, 2023.

OWASP FOUNDATION. OWASP Top 10:2021. 2021.

OWASP GENAI SECURITY PROJECT. OWASP Top 10 for LLM Applications 2025. 2024.

PEARCE, H. et al. Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. In: 43rd IEEE Symposium on Security and Privacy (SP), 2022, p. 754–768. DOI: 10.1109/SP46214.2022.9833571.

PERRY, N. et al. Do users write more insecure code with AI assistants? In: ACM SIGSAC Conference on Computer and Communications Security (CCS '23), 2023, p. 2785–2799. DOI: 10.1145/3576915.3623157.

SIDDIQ, M. L.; SANTOS, J. C. S. SecurityEval dataset: mining vulnerability examples to evaluate machine learning-based code generation techniques. In: MSR4P&S '22, 2022, p. 29–33. DOI: 10.1145/3549035.3561184.

SNYK. AI Code Security Report. 2023.

SPRACKLEN et al. We have a package for you! A comprehensive analysis of package hallucinations by code generating LLMs. In: USENIX Security Symposium, 2025, p. 3687–3706.

VERACODE. 2025 GenAI Code Security Report. 2025.